PHISHGUARD

AUTOMATED PHISHING DETECTION SYSTEM

System Overview & Technical Documentation

Version	1.0
Date	May 2026
Instructors	Dr. Taghreed Salem Dr. Mariam Mohamed
Classification	Confidential — *INSIDEREYE_TEAM*
Purpose	Educational, Security Research & Threat Mitigation
AI Engine	Keras / TensorFlow Deep Learning Neural Network
Contribution	Ahmed Eldesouky Ahmed Elasmar Hazem Ashraf Ahmed Abdelmoniem

1. Executive Summary 3
2. System Architecture 4
2.1 Background IMAP Daemon 5
2.2 Flask Backend API 5
2.3 Web Dashboard (Frontend) 5
3. Core Features 6
3.1 Real-Time Inbox Monitoring 6
3.2 Dual-Engine Detection 7
3.3 Explainable AI Verdicts 7
4. Detection Engines 8
4.1 Rule-Based Heuristic Engine 8
4.2 Deep Learning Neural Network 8
5. AI Model — Training & Architecture 10
5.1 Dataset Preparation 10
5.2 Model Architecture 10
5.3 Training Framework 10
6. Model Performance & Analytics 11
7. System Workflow 12
8. Technical Stack 13
9. Conclusion 14

1. Executive Summary

PhishGuard is an advanced, end-to-end automated phishing email detection system that bridges the gap between passive email filtering and proactive user alerting. It monitors a user's inbox in real-time, analyzes every incoming email using a dual-engine approach, and instantly alerts the user of potential threats via desktop notifications — before the user interacts with a malicious message.

The system's core philosophy is a "dual-engine defense": no single method is relied upon alone. A traditional Rule-Based heuristic engine handles known indicators quickly, while a Deep Learning Neural Network provides probabilistic, context-aware classification that generalizes to zero-day threats and previously unseen phishing campaigns.

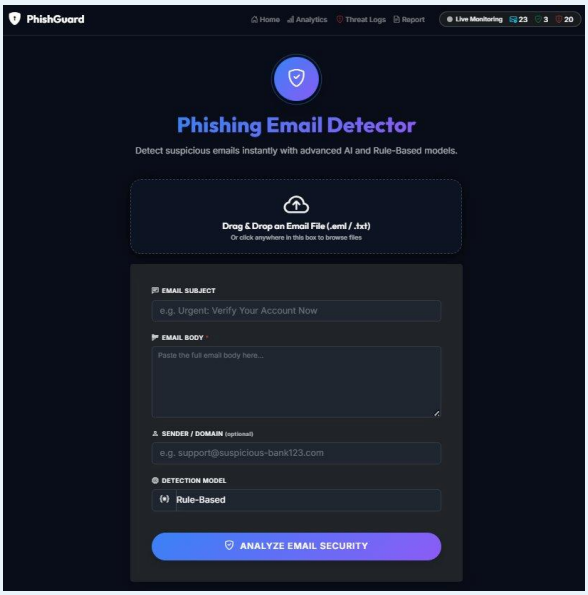

Figure 1.1 — Main Dashboard Overview

Figure 1.1 — The PhishGuard home dashboard

Primary Purpose	Real-time phishing email detection and proactive user alerting
Target Users	Individual users, security researchers, IT administrators
Deployment Model	Local daemon + Flask backend + Dark-themed web dashboard
Detection Engines	Rule-Based Heuristics + Keras Deep Learning Neural Network
AI Accuracy	Neural Network: ~92.33% accuracy, 90.71% F1-Score
Alert Mechanism	Windows Desktop Toast Notification (triggered within 15 seconds)
Email Protocol	IMAP over SSL (Port 993) for secure inbox monitoring
Reporting	Historical Threat Intelligence Logs with risk scores and model attribution

2. System Architecture

PhishGuard is organized into three primary, clearly separated components. Each has a distinct responsibility, and they communicate via a secure local HTTP interface.

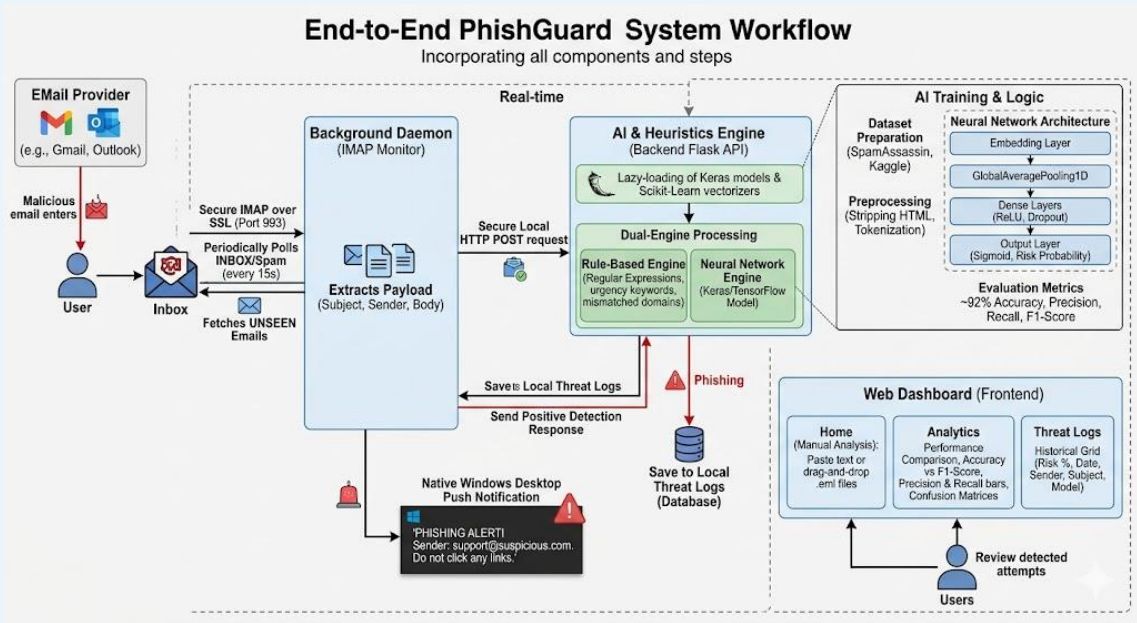

Figure 2.1 — System Architecture Diagram

Figure 2.1 — High-level architecture: IMAP Daemon → Flask Backend API → Web Dashboard, showing data flow between all three components.

Component	Description & Key Responsibilities
Background IMAP Daemon	Runs autonomously in the background. Connects to the inbox via IMAP over SSL, polls every 15 seconds for UNSEEN emails, extracts payloads, and dispatches them to the Flask API. Fires Windows toast notifications on threat detection.
Flask Backend API	The analysis brain. Exposes /analyze and /inbox-stats endpoints. Lazy-loads Keras models and Scikit-Learn vectorizers. Routes every email through both detection engines and returns a JSON verdict with risk score and explanation.
Web Dashboard (Frontend)	Dark-themed HTML5/CSS3/JS interface. Provides manual email analysis (drag-and-drop), an Analytics page with comparative model metrics, and a Threat Intelligence Logs page for reviewing historical detections.

2.1 Background IMAP Daemon

The daemon (imap_monitor.py) operates headlessly in the background with no user interaction required after startup. Its key behaviors:

- Connects securely to the mail provider (Gmail, Outlook, etc.) using IMAP over SSL on port 993
- Polls both INBOX and Spam folders every 15 seconds based on a configurable interval
- Fetches only UNSEEN (unread) emails to optimize bandwidth and avoid reprocessing
- Extracts the email Subject, Sender (From field), and fully decodes the multipart Body payload
- Sends a local HTTP POST to http://localhost:5000/analyze with the extracted data
- On a “Phishing” verdict: fires a native Windows Desktop Toast Notification immediately

> **15-Second Detection Window**
> From the moment a phishing email lands in the inbox, the daemon detects and alerts the user within 15 seconds — before the user has a chance to open, click, or interact with any malicious content.

2.2 Flask Backend API

The backend (app.py) is the central analysis hub. It exposes two RESTful endpoints:

- /analyze — Receives the email payload (subject, sender, body), routes it through both detection engines, and returns a JSON response containing the verdict, risk score, triggered rules, and a human-readable explanation.
- /inbox-stats — A polling endpoint that provides the frontend with real-time counts of scanned emails and detected threats for live dashboard updates.

Memory management uses lazy-loading: the heavy Keras (.keras) model and Scikit-Learn TF-IDF vectorizer are only loaded into RAM on the first /analyze request. This minimizes startup time while keeping subsequent analyses fast.

2.3 Web Dashboard (Frontend)

The dashboard runs on top of the Flask server and provides three primary pages:

- Home — Manual analysis interface. Users can paste raw email text or drag-and-drop .eml files for instant on-demand analysis. Displays the risk score, verdict, and triggered rules.
- Analytics — A data-driven comparison page showing Accuracy, F1-Score, Precision, Recall bar charts, and dynamic Confusion Matrices for both the Rule-Based and Neural Network engines side by side.
- Threat Logs — A historical grid view of all previously detected phishing attempts. Each card shows the risk percentage, detection timestamp, sender, subject, and which model flagged the email.

3. Core Features

PhishGuard delivers seven integrated capabilities that together form a complete phishing defense and analysis platform:

3.1 Real-Time Inbox Monitoring

The daemon continuously monitors the inbox without any manual triggering. Every new unread email is intercepted, analyzed, and classified within the 15-second polling window. The user is alerted before they can interact with any malicious content.

Figure 3.1 — Real-Time Monitoring

Figure 3.1 — The Threat Logs page showing historical phishing attempts as grid cards with risk scores, timestamps, senders, and model attribution

3.2 Dual-Engine Detection

Every email is evaluated by both engines independently. Their verdicts are combined into a final consensus score:

- Rule-Based Engine — Fast pattern matching using strict regular expressions. Effective for well-known phishing templates but limited on novel attacks.
- Neural Network Engine — Context-aware deep learning classification. Generalizes to previously unseen phishing campaigns by understanding semantic patterns in the email body.

> ⚠ **Why Two Engines?**
> The Rule-Based engine has high precision but low recall (misses novel attacks). The Neural Network has high recall but may occasionally flag ambiguous legitimate emails. Together, their consensus minimizes both false positives and false negatives.

3.3 Explainable AI Verdicts

PhishGuard never delivers an unexplained verdict. Every classification is accompanied by a human-readable explanation detailing:

- Which specific rules were triggered (Rule-Based engine)
- The exact risk probability score from the Neural Network (0% to 100%)
- The primary reason for the classification (urgency keyword, suspicious URL, sender mismatch, etc.)

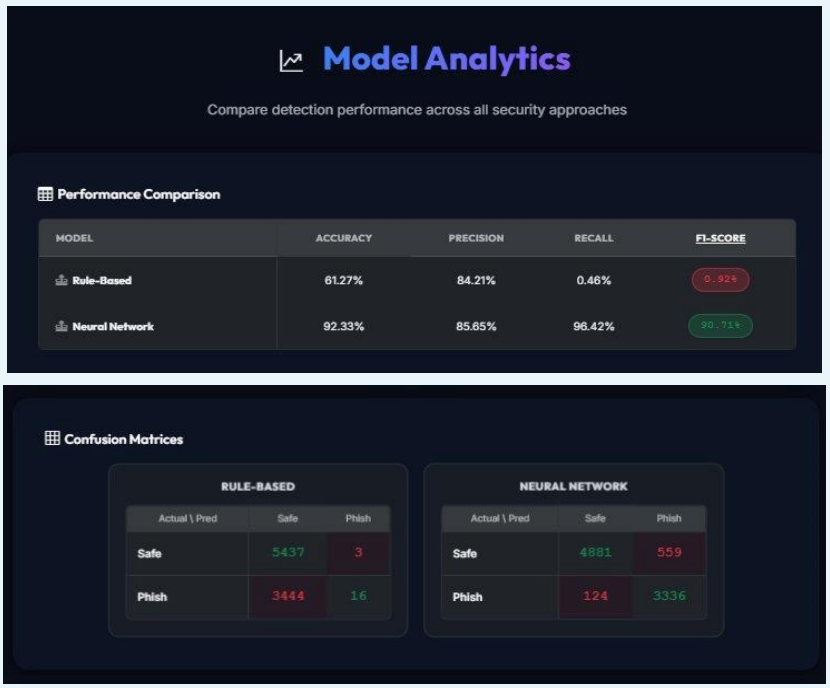

Figure 3.2 — Analytics Dashboard

Figure 3.2 — The Analytics page showing comparative Accuracy, F1-Score, Precision, Recall metrics, and Confusion Matrices for both detection models.

4. Detection Engines

Each engine operates independently and contributes a distinct signal to the final verdict. The engines are designed to complement each other's weaknesses.

4.1 Rule-Based Heuristic Engine

This engine applies strict Regular Expressions and string-matching algorithms to detect established phishing indicators without any model loading overhead. It is the first line of defense due to its speed:

- Urgency Keywords — detects phrases like "Verify your account", "Suspended", "Immediate action required", "Your account will be closed"
- Suspicious Links — flags IP addresses used in URLs instead of domain names, mismatching anchor text vs. href targets, and known URL shortener services
- Sender Discrepancies — detects mismatches between the display name and the actual SMTP envelope sender (e.g., display: "PayPal Support", actual: support@suspicious-domain.ru)
- Domain Spoofing — identifies look-alike domains using character substitution (e.g., paypa1.com, g00gle.com)

Strength	Limitation
Zero latency — no model loading	Fails on novel/obfuscated phishing
High precision on known templates	Very low recall on zero-day attacks
Human-auditable rule triggers	Cannot understand semantic context
Works offline, no GPU needed	Requires constant manual rule updates

4.2 Deep Learning Neural Network

The primary classification engine is a Keras/TensorFlow Neural Network that understands the semantic context of the email payload — not just surface patterns. It is loaded lazily on the first request to minimize startup time.

The model processes email text through a custom TextVectorization layer that removes HTML tags, punctuation, and stop words, then converts words into integer sequences. These sequences are fed into the following architecture:

- Embedding Layer — converts integer sequences into dense semantic vectors, allowing the network to learn that "urgent" and "immediate" have similar threat relevance
- GlobalAveragePooling1D — flattens the embedding output and reduces dimensionality, preventing positional overfitting
- Dense Layers with ReLU — multi-layer perceptron blocks that learn complex, non-linear patterns in email structure
- Dropout Layers (20%) — randomly drops connections during training to force generalization rather than memorization

- Sigmoid Output Layer — produces a continuous float in [0.0, 1.0] representing the Risk Probability

Strength	Limitation
Generalizes to unseen phishing styles	Requires training data and GPU resources
High recall — catches zero-day threats	Slower first-load due to model initialization
Understands semantic context	Less interpretable than rule triggers
Continuously improvable via retraining	Needs balanced, high-quality training data

Combined Verdict Logic

When both engines agree on a phishing verdict, confidence is very high. When they disagree (e.g., Rule-Based: Clean, Neural Network: 78% risk), the Neural Network score is weighted more heavily due to its superior recall on novel threats.

5. AI Model — Training & Architecture

5.1 Dataset Preparation

The Neural Network is trained on a carefully prepared dataset to ensure generalization across diverse phishing styles:

- Primary Source — SpamAssassin Public Corpus: the industry-standard benchmark dataset for email classification
- Supplementary Source — Modern phishing datasets from Kaggle covering recent social engineering campaigns
- Balancing — the dataset is carefully balanced between “Ham” (safe) and “Phishing” emails to prevent class bias
- Preprocessing — HTML tags, punctuation, and stop words are stripped. Text is lowercased and tokenized.

5.2 Model Architecture

Layer	Type / Config	Purpose
Input	TextVectorization	Converts raw email text to integer token sequences
Embedding	Dense vector, fixed size	Maps tokens to semantic vectors — similar words cluster together
GlobalAvgPool1D	Pooling	Flattens embeddings, reduces overfitting on word positions
Dense (Hidden 1)	ReLU activation	Learns complex non-linear phishing patterns
Dropout	20% drop rate	Prevents memorization, forces generalization
Dense (Hidden 2)	ReLU activation	Refines learned features for final classification
Output	Sigmoid activation, 1 unit	Outputs Risk Probability: 0.0 (Safe) — 1.0 (Phishing)

5.3 Training Framework

Framework	Keras / TensorFlow
Input Format	Raw email text strings (pre-processed by TextVectorization layer)
Output	Risk probability float [0.0 – 1.0] → converted to percentage (0% – 100%)
Vectorizer	Scikit-Learn TF-IDF (also used by Rule-Based; saved as .pkl for lazy-loading)
Model File	Saved as .keras format for efficient deployment

6. Model Performance & Analytics

The Analytics dashboard provides a clear, data-driven comparison of both detection models. The Neural Network significantly outperforms the Rule-Based engine, particularly on the Recall metric which is critical for catching zero-day phishing attempts.

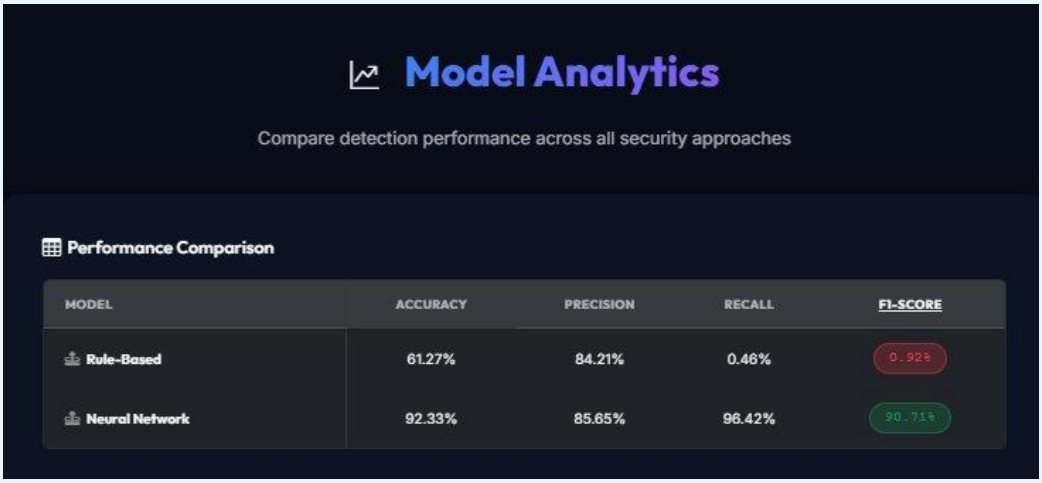

Figure 6.1 — Analytics Dashboard — Model Comparison

Figure 6.1 — Comparative metrics: Accuracy, F1-Score, Precision, Recall bar charts, and dynamic Confusion Matrices for both models.

Metric	Neural Network	Rule-Based Engine
Accuracy	~92.33%	~61.27%
F1-Score	~90.71%	Low (due to poor Recall)
Precision	High	High (strict rules = fewer false positives)
Recall	High	Low (misses novel/obfuscated phishing)

The Confusion Matrices track four outcomes for continuous monitoring:

- True Positives (TP) — phishing emails correctly identified as phishing
- False Positives (FP) — legitimate emails incorrectly flagged as phishing (minimized by Precision)
- True Negatives (TN) — safe emails correctly passed through
- False Negatives (FN) — phishing emails missed by the model (minimized by Recall)

Why Recall Matters Most

In phishing detection, a False Negative (missed phishing email) is far more dangerous than a False Positive (blocking a safe email). The Neural Network's high Recall ensures that the vast majority of phishing attempts are caught, even if occasionally a legitimate email is reviewed.

7. System Workflow

From startup to notification, PhishGuard operates through a deterministic 9-step pipeline:

Step	Stage	What Happens
1	**Initialization**	User starts Flask server (app.py) and IMAP daemon (imap_monitor.py). Dashboard becomes accessible at http://localhost:5000.
2	**Monitoring**	Daemon silently connects to inbox via IMAP SSL and begins polling INBOX + Spam folders every 15 seconds.
3	**Interception**	A new unread email arrives. The daemon detects it within the next polling cycle.
4	**Extraction**	Subject, Sender, and decoded Body payload are extracted from the email. Multipart MIME is fully decoded.
5	**API Dispatch**	Extracted data is sent via HTTP POST to http://localhost:5000/analyze.
6	**Dual Classification**	Flask routes the payload through the Rule-Based engine first, then the Neural Network. Both produce independent scores.
7	**Logging**	The threat is saved to the local database, making it visible on the Threat Logs dashboard page.
8	**Alerting**	The Flask API responds to the daemon with a “Phishing” verdict.
9	**Notification**	The daemon fires a Windows toast: "PHISHING ALERT! Sender: support@suspicious.com. Do not click any links."

8. Technical Stack

The following technologies form the complete software stack of PhishGuard:

Category	Technology	Purpose & Notes
Backend	**Python 3.10+**	All analysis logic, daemon, and orchestration
Web Framework	**Flask**	RESTful API endpoints and web dashboard server
AI Framework	**Keras / TensorFlow**	Neural Network model definition, training, and inference
Vectorization	**Scikit-Learn (TF-IDF)**	Feature extraction for Rule-Based engine; lazy-loaded .pkl
Text Processing	**Keras TextVectorization**	Custom tokenizer for Neural Network input pipeline
Email Protocol	**IMAP over SSL (993)**	Secure inbox monitoring — imaplib Python standard library
Notifications	**Windows Toast API**	Native desktop push notifications via win10toast or plyer
Frontend	**HTML5 + CSS3 + JS**	Vanilla stack — no heavy framework, fast load, dark-themed
Database	**Local file / SQLite**	Persistent threat intelligence log storage
Model Storage	**.keras + .pkl files**	Serialized model and vectorizer for lazy deployment

9. Conclusion

PhishGuard successfully bridges the gap between passive email filtering and proactive, real-time user alerting. By leveraging advanced Deep Learning alongside traditional heuristics, it provides a highly reliable, explainable, and fast defense mechanism against sophisticated social engineering and phishing attacks.

The system's dual-engine architecture ensures that:

- Known phishing templates are caught immediately by the Rule-Based engine with zero model overhead
- Novel, zero-day phishing campaigns are identified by the Neural Network with ~92.33% accuracy
- Every verdict is explainable — users and analysts understand exactly why an email was flagged
- Real-time alerting means users are warned within 15 seconds, before any interaction with malicious content
- Historical threat logs enable continuous monitoring, pattern analysis, and model improvement

Future development directions may include:

- Multi-platform notification support (macOS, Linux)
- URL sandboxing integration for live link analysis
- BERT or transformer-based model for higher semantic understanding
- Browser extension for webmail integration (Gmail/Outlook web)
- SMTP gateway mode for organization-wide deployment